

MATA KULIAH • MACHINE LEARNING

Data Preparation Lanjutan

Missing Values • Outlier • Imbalanced Data

Studi Kasus: Prediksi Telco Customer Churn

KELOMPOK 3

Machine Learning

1. Frisilia Elfira Maharani
2. Febyana Mulia Putri
3. Wanda Nurhasanah
4. Muhammad Revolvere Rey A
5. Ahmad Dandi Subhani

Tujuan Pembelajaran

Setelah presentasi ini, kita mampu ...

1. Mendiagnosis missing values

dan memilih strategi imputasi yang sesuai.

2. Mendeteksi serta menilai outlier

sebelum memutuskan menghapus atau mempertahankannya.

3. Menangani imbalanced data

dengan SMOTE secara benar (hanya pada data train).

4. Membangun alur preprocessing yang aman

dan mengevaluasi model dengan metrik yang jujur.

Dataset: Telco Customer Churn

Data nyata 7.043 pelanggan layanan telekomunikasi

Target: Churn (Yes / No)

Apakah pelanggan berhenti berlangganan?

21 kolom:

Demografi (gender, SeniorCitizen, Partner, Dependents), layanan (Phone, Internet, OnlineSecurity, ...), kontrak & pembayaran, tagihan (tenure, MonthlyCharges, TotalCharges).

Fakta awal:

- 0 data duplikat — data bersih dari sisi duplikasi.
- TotalCharges terbaca sebagai teks karena ada 11 sel kosong (spasi).
- Churn = Yes hanya 26,5% → data tidak seimbang.

Tiga Masalah Data, Tiga Keputusan

Temuan pada dataset Telco Churn

MISSING VALUES

11 baris kosong di TotalCharges.

Ternyata: pelanggan baru (tenure = 0) — tagihannya memang belum ada.

Keputusan: imputasi, bukan hapus.

OUTLIER

113 baris (2%) terdeteksi Isolation Forest.

Ternyata: pelanggan tenure sangat panjang — kejadian sah.

Keputusan: dipertahankan, bukan dibuang.

IMBALANCED DATA

Churn Yes : No = 26,5% : 73,5%.

Risiko: model "malas", selalu menebak Tidak Churn.

Keputusan: SMOTE hanya pada data train.

Aturan Aman Preprocessing

Agar model tidak "mencontek" dari data test

1. SPLIT

train & test
(stratified)
— paling
awal

2. FIT

imputer &
scaler
hanya
belajar dari
train

3. TRANSFORM

terapkan ke
train & test

4. RESAMPLE

SMOTE
hanya pada
train

5. EVALUASI

pada test
asli yang
tidak
seimbang

Mean, median, tetangga KNN, dan sampel sintetis TIDAK BOLEH memperoleh informasi dari test set.

Missing Values → KNN Imputer

Ada informasi yang tidak tercatat — hilang acak atau berpola?

Temuan: 11 missing di TotalCharges = pelanggan baru (tenure 0). Ini berpola (MAR), bukan acak.

Menghapus barisnya = membuang segmen pelanggan baru → bias.

Solusi: KNN Imputer (k=5)

Menebak nilai kosong dari 5 pelanggan paling mirip (tenure, MonthlyCharges, ...).

Fit di train → transform train & test. Hasil: 0 missing tersisa.

Analogi sederhana: nilai rapor yang hilang ditebak dari nilai teman yang paling mirip.

Outlier → Dinilai, Lalu Dipertahankan

Nilai jauh dari mayoritas: kesalahan data atau kejadian langka yang sah?

Deteksi: Isolation Forest (contamination=2%) → 113 baris outlier di data train.

Analisis: rata-rata tenure outlier ± 48 bulan vs ± 32 bulan normal — mereka pelanggan setia lama.

Keputusan: DIPERTAHANKAN.

Menghapusnya = membuang pelanggan paling loyal, justru segmen penting untuk analisis churn.

Pelajaran: tidak ada teknik yang selalu benar — keputusan mengikuti konteks domain.

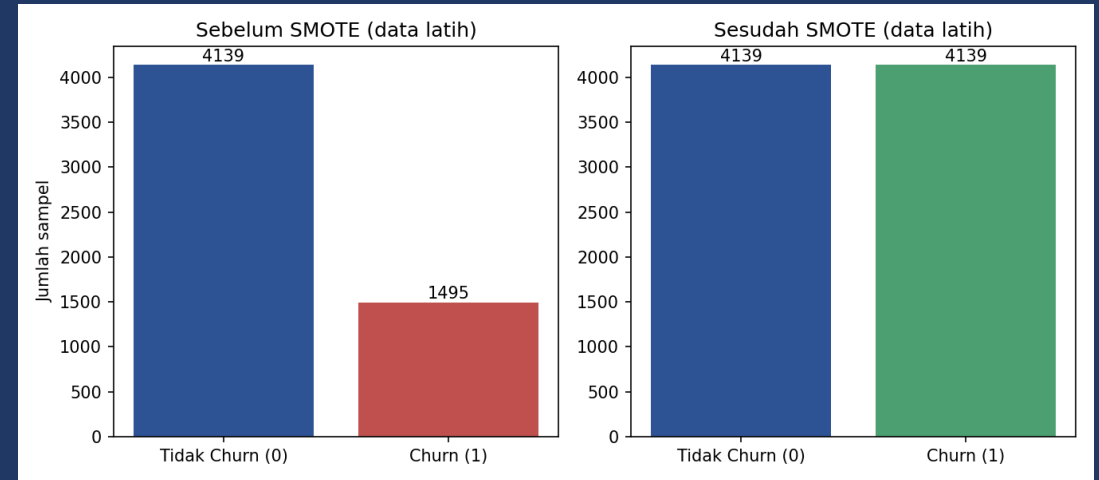
Imbalanced Data → SMOTE

Satu kelas jauh lebih sedikit — apakah model mengabaikan kelas penting?

Distribusi train: 4.139 vs 1.495 (26,5% churn).

Tanpa penanganan, model bisa mencapai akurasi "bagus" hanya dengan selalu menebak kelas mayoritas — tapi gagal total mendeteksi churn.

Solusi: SMOTE membuat sampel sintetis kelas minoritas hingga seimbang (4.139 : 4.139) — HANYA pada data train.

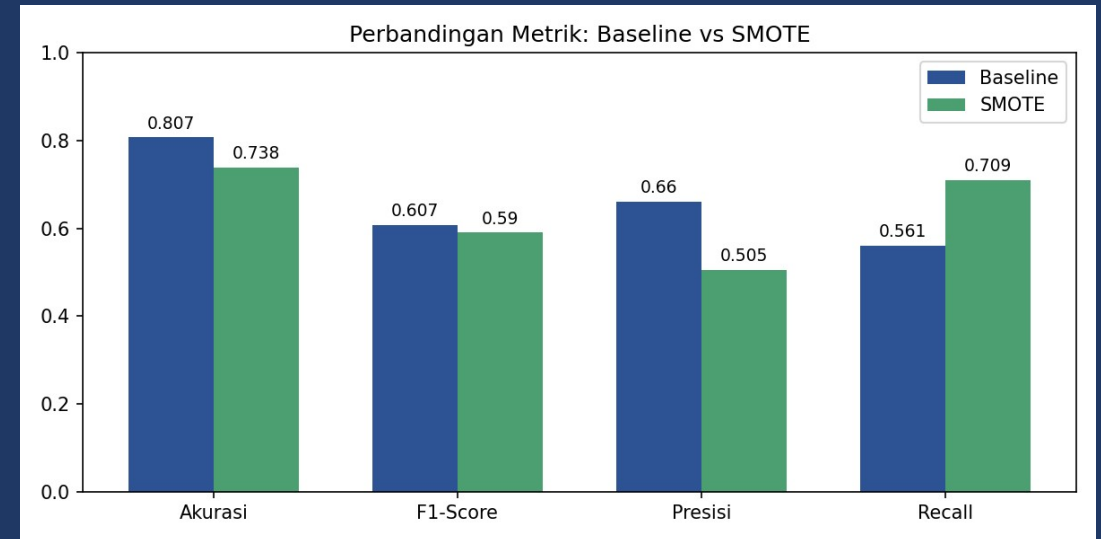


Hasil: Baseline vs SMOTE

Dievaluasi pada 1.409 data test ASLI (tidak seimbang)

Model	Akurasi	F1	Presisi	Recall
Baseline	0.807	0.607	0.66	0.561
SMOTE	0.738	0.59	0.505	0.709

SMOTE menaikkan recall 0.56 → 0.71 (menangkap lebih banyak churn), tapi precision turun 0.66 → 0.51 (lebih banyak false alarm).



Bedah Confusion Matrix

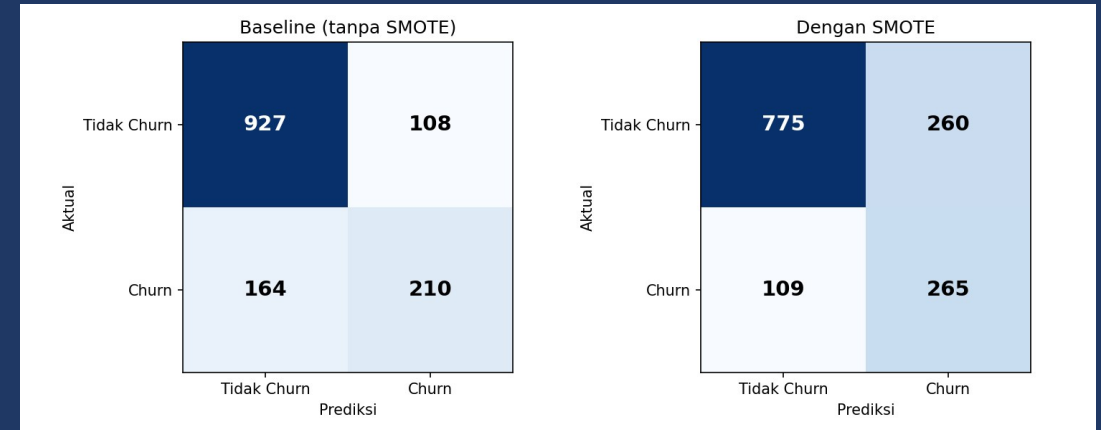
Angka di balik metrik — pada 374 pelanggan yang benar-benar churn

Baseline: 210 churn tertangkap, 164 lolos diam-diam.

SMOTE: 265 churn tertangkap, hanya 109 yang lolos.

Harga yang dibayar: false alarm naik dari 108 → 260.

Untuk bisnis, melewati churn (lolos diam-diam) lebih mahal daripada salah mengingatkan — maka recall lebih penting.



Bedah Kode (1): Split & Imputasi

Preprocessing yang aman — test tidak boleh bocor ke proses belajar

```
# 1. Split DULU — paling awal (stratified)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

# 2. KNN Imputer — hanya belajar dari train
imputer = KNNImputer(n_neighbors=5)
X_train[num_cols] =
imputer.fit_transform(X_train[num_cols])
X_test[num_cols] = imputer.transform(X_test[num_cols])

# 3. Scaling — hanya belajar dari train
scaler = StandardScaler()
X_train[num_cols] =
scaler.fit_transform(X_train[num_cols])
X_test[num_cols] = scaler.transform(X_test[num_cols])
```

train_test_split: bagi 80% latih / 20% uji;
stratify menjaga proporsi churn di
kedua sisi.

**Pola fit_transform → transform
adalah inti "aturan aman".**

KNNImputer(k=5): 11 nilai kosong
ditebak dari 5 pelanggan paling mirip.

StandardScaler: menyamakan skala
angka agar model konvergen.

Bedah Kode (2): Outlier & SMOTE

Deteksi anomali, lalu seimbangkan kelas — keduanya hanya di train

```
# 4. Deteksi outlier – di train saja
iso = IsolationForest(contamination=0.02,
random_state=42)
is_outlier = iso.fit_predict(X_train) # -1 = outlier
# → 113 baris (2%): tenure panjang → DIPERTAHANKAN

# 5. SMOTE – HANYA pada train!
smote = SMOTE(random_state=42)
X_train_sm, y_train_sm = smote.fit_resample(X_train,
y_train)
# → 4139 : 4139, kelas jadi seimbang
```

IsolationForest memisahkan 2% data paling "aneh" (label -1).

Keputusan domain: outlier = pelanggan loyal, bukan error — jangan dihapus.

SMOTE membuat sampel sintetis kelas minoritas hingga seimbang.

Hanya di train — kalau di test, evaluasi jadi tidak jujur.

Bedah Kode (3): Training & Evaluasi

Algoritma sama, data latih beda — diuji di test asli

```
# 6. Latih dua model
model_base =
LogisticRegression(max_iter=2000).fit(X_train, y_train)
model_smote =
LogisticRegression(max_iter=2000).fit(X_train_sm,
y_train_sm)

# 7. Evaluasi — selalu di test ASLI
y_pred = model.predict(X_test)
accuracy_score(y_test, y_pred) # 0.807 vs 0.738
f1_score(y_test, y_pred)      # 0.607 vs 0.590
```

max_iter=2000 memastikan optimasi konvergen.

Bedanya hanya data latih: SMOTE vs tidak.

Evaluasi di test asli (tidak seimbang) → hasil jujur seperti kondisi nyata.

SMOTE menang recall (0.71 vs 0.56): lebih banyak churn tertangkap.

Kesimpulan

Mengubah data bermasalah menjadi masukan model yang dapat dipercaya

1. Diagnosis dulu, teknik kemudian.

11 missing ternyata pelanggan baru → imputasi KNN. 113 outlier ternyata pelanggan loyal → dipertahankan.

2. Aturan aman tidak bisa ditawar.

Split → fit di train → SMOTE hanya di train → evaluasi di test asli.

3. Akurasi bisa menipu.

Baseline "terlihat" menang (0.807 vs 0.738), padahal melewatkan lebih banyak churn. Pilih metrik sesuai tujuan: recall untuk kasus churn.

4. Tidak ada teknik yang selalu benar.

Setiap keputusan mengikuti pola data dan konteks domain.

Terima Kasih

Kelompok 3 — Machine Learning

Frisilia Elfira Maharani • Febyana Mulia Putri • Wanda Nurhasanah
Muhammad Revolvere Rey A • Ahmad Dandi Subhani